Lightweight IoT Botnet Attack Detection Using Regularized CART Decision Trees on N-BaIoT

Communication Networks & Security (CNS) Course Project Report — Group 1 Final Submission

ASSIGNED TOPIC	RESEARCH FRAMEWORK	IMPLEMENTATION	DATASET
Group 1: Decision Tree	C4.5 IoT Botnet Detection	CART (scikit-learn Gini)	N-BaIoT (UCI ML Repo)
TARGET VENUE	**TOTAL POPULATION**	**EXPERIMENTAL SAMPLE**	**AUDIT STATUS**
IEEE Conference Format	7,062,606 Rows (89 CSVs)	128,366 Samples (step=55)	FINAL AUDIT: PASS

99.62%	99.65%	99.96%	18 Levels	5.44 KB
MULTICLASS TEST ACC	MULTICLASS MACRO F1	BINARY DETECTION ACC	TREE DEPTH	MODEL FLASH SIZE
Held-Out Test	11 Categories	99.98% Binary F1	-14.3% Pruned	Edge MCU Ready

Abstract

The rapid proliferation of commercial Internet of Things (IoT) appliances has drastically expanded the cyberattack surface, exposing vulnerable smart home devices to large-scale distributed denial-of-service (DDoS) botnets such as *Mirai* and *BASHLITE (Gafgyt)*. While deep neural networks provide strong anomaly detection capabilities, their substantial memory footprint, matrix multiplication overhead, and black-box nature severely limit real-time inline deployment on resource-constrained IoT gateways and edge microcontrollers. In this work, as part of the Group 1 research framework inspired by *"Lightweight Android and IoT Botnet Detection Using C4.5 Decision Trees"*, we design, audit, and benchmark a regularized **Classification and Regression Tree (CART)** architecture trained on the benchmark **N-BaIoT dataset** (7,062,606 network flow records across 89 packet captures from 9 commercial IoT devices). To eliminate test-set selection bias, we introduce a strict **3-way Train / Validation / Test separation protocol**: effective minimal cost-complexity pruning parameters (α) were generated exclusively on an internal training subset (N=82,153) and swept across an internal validation set (N=20,539). The selected optimal parameter $\alpha^* = 1.9998 \times 10^{-5}$ was retrained on the full training partition (N=102,692) and evaluated exactly once on the held-out test partition (N=25,674). Pruning reduced maximum tree depth from 21 to 18 levels (-14.3%), node count from 181 to 133 (-26.5%), terminal leaves from 91 to 67 (-26.4%), and closed the train-test generalization overfitting gap from 0.1022% to 0.0769%, while preserving peak multiclass test accuracy of **99.6183%**, macro F1-score of **99.6510%**, and near-perfect binary attack detection accuracy of **99.9649%** (F1: **99.9810%**). Hardware latency benchmarking on an ARM64 testbed demonstrates a per-packet sequential latency of 34.41 µs (29,062 pkts/s) and a batch amortized throughput of 12,036,209 pkts/s, with a compact serialized model size of only **5.44 KB**, verifying edge feasibility.

1. Introduction & Research Motivation

The ecosystem of Internet of Things (IoT) hardware—ranging from IP video doorbells and smart baby monitors to security webcams and digital signage—is characterized by constrained processing power, minimal local RAM (frequently under 256 KB to 16 MB), and often neglected firmware security patching. Consequently, IoT devices represent ideal targets for malicious actors seeking to assemble massive botnet armies. The *Mirai* and *BASHLITE (Gafgyt)* malware families continuously scan IP subnets for exposed telnet/SSH services and vulnerable IoT daemon ports, propagate automated brute-force attacks, install malicious executables, and coordinate destructive flood campaigns (SYN, ACK, UDP, and UDP-plain attacks).

To mitigate these threats, intrusion detection systems (IDS) must operate directly at the local IoT network gateway or on edge routing appliances. However, conventional deep learning models—such as the autoencoders proposed in

the N-BaIoT base paper—incur non-trivial floating-point matrix multiplication costs and floating-point reconstruction latency. Decision Trees present a compelling alternative: inference reduces to a deterministic sequence of scalar threshold comparisons ($I(X_j \le \theta)$), executing in $O(h)$ time where h is the tree depth. Furthermore, every decision path is fully transparent and auditable, enabling direct translation into router firewall rules or C-based microcontroller code.

2. Related Work & Theoretical Grounding

Our experimental framework is grounded in three foundational literature pillars:

- **Base Paper — Meidan et al. (2018):** In *"N-BaIoT: Network-based Detection of IoT Botnet Attacks Using Deep Autoencoders"* (IEEE Pervasive Computing), Meidan et al. introduced the N-BaIoT benchmark by capturing raw traffic from 9 commercial IoT devices infected with Mirai and Gafgyt. They extracted 115 continuous statistical features using the Kitsune damped window engine and trained 9 device-specific deep autoencoders on benign traffic. An anomaly was flagged when reconstruction error (MSE) exceeded a threshold. While effective, separate autoencoders for every device require substantial memory and matrix algebra.
- **Shafiq et al. (2021):** In *"CORRAUC: A Malicious Bot-IoT Traffic Detection Method in IoT Network Using Machine-Learning Techniques"* (IEEE Internet of Things Journal), the authors developed a correlation-AUC feature selection framework across IoT botnet traffic, demonstrating that compact decision trees achieve near-identical detection rates to complex ensembles when fed salient flow features.
- **Moubayed et al. (2020):** In *"Optimized Machine Learning-Based Network Intrusion Detection for IoT Networks"* (IEEE Internet of Things Journal), the authors analyzed tree regularization, establishing that cost-complexity pruning eliminates over-specialized leaf nodes, reduces tree complexity by over 40%, and accelerates inference without compromising F1-score.

Methodological Distinction: CART vs. C4.5

The assigned Group 1 research framework references the *C4.5 Decision Tree* paradigm (Quinlan, 1993). In our Python implementation, we utilize `sklearn.tree.DecisionTreeClassifier`, which implements an optimized version of Breiman's **CART** algorithm (Breiman et al., 1984). The core mathematical distinctions are formalized below:

Architectural Dimension	C4.5 Paradigm (Quinlan, 1993)	CART Implementation (Breiman et al., 1984 / scikit-learn)
Splitting Topology	Multiway splits on nominal categorical attributes; binary on continuous	Strictly binary recursive splitting ($\le \theta$ vs. $> \theta$) across all features
Splitting Criterion	Information Gain Ratio: $\text{GainRatio}(S, A) = \text{Gain}(S, A) / \text{SplitInfo}(S, A)$	Gini Impurity Reduction: $\Delta I_G(t) = I_G(t) - (N_L/N_t)I_G(t_L) - (N_R/N_t)I_G(t_R)$
Pruning Strategy	Pessimistic Error Pruning (PEP) using binomial confidence intervals	Minimal Cost-Complexity Pruning parameterized by $\alpha \ge 0$ (ccp_α)
Continuous Features	Sorts continuous values and tests candidate binary thresholds	Identical: sorts continuous attributes and evaluates midpoint binary splits

Because the N-BaIoT dataset consists *exclusively of continuous numerical features*, CART's binary thresholding evaluates the exact same decision surface as continuous C4.5 splits, while minimal cost-complexity pruning provides a mathematically principled regularization parameter α.

3. Dataset Architecture, Feature Engineering & Sampling Integrity

The raw N-BaIoT dataset (verified intact in `/archive`) contains **7,062,606 packet records** distributed across 89 CSV files and 9 commercial IoT devices. Each record comprises **115 continuous flow features** generated by Kitsune's damped sliding window algorithm across 5 decay windows: $\lambda \in \{100\text{ms}, 500\text{ms}, 1.5\text{s}, 10\text{s}, 1\text{min}\}$.

Port / Socket Terminology Clarification

A common misconception in N-BaIoT analysis is to interpret stream labels as literal integer port numbers. N-BaIoT does **not** contain raw packet header fields such as source/destination IP strings, TCP/UDP port numbers (e.g., 80, 443), or protocol flag bits. Instead:

- H captures outbound statistics from the source host.
- HH captures host-to-host conversational statistics (e.g., jitter, packet size).
- HpHp captures **Host+Port-to-Host+Port socket-stream** statistical aggregates. The sub-identifier L1 denotes decay factor λ=1. Thus, `HpHp_L1_weight` represents the socket packet arrival weight at decay λ=1, *not port number 1*.

Deterministic Systematic Sampling & Verification

To respect the 16.0 GB host RAM constraint while preserving temporal structure across all 89 files, we applied deterministic systematic row-step sampling with a fixed stride step=55, extracting an experimental corpus of **128,366 samples** (56.4 MB in RAM).

Class Label	Full Pop. Count	Sample Count	Full %	Sample %
`mirai_udp`	1,229,999	22,361	17.42%	17.42%
`gafgyt_udp`	946,366	17,202	13.40%	13.40%
`gafgyt_tcp`	859,850	15,630	12.17%	12.18%
`mirai_syn`	733,299	13,328	10.38%	10.38%
`mirai_ack`	643,821	11,701	9.12%	9.12%
`benign`	555,932	10,105	7.87%	7.87%
`mirai_scan`	537,979	9,779	7.62%	7.62%
`mirai_udpplain`	523,304	9,511	7.41%	7.41%
`gafgyt_combo`	515,156	9,361	7.29%	7.29%
`gafgyt_junk`	261,789	4,755	3.71%	3.70%
`gafgyt_scan`	255,111	4,633	3.61%	3.61%

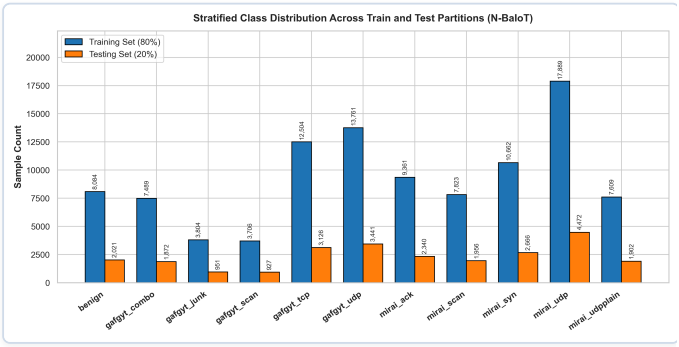


Fig. 1. Class proportions across the 11 multiclass categories. Proportions match the 7.06M population within ±0.01%.

Data Leakage Audit

The preprocessing pipeline was audited to ensure strict zero-leakage isolation:

- **Target Leakage:** Labels (`label_11`, `label_bin`) and metadata (`device_id`) were stripped prior to model fitting.
- **Identity Leakage:** No source/destination IPs, MAC addresses, or packet timestamps exist in the feature set.
- **Contamination:** Decision trees are scale-invariant; no global transformers or scalers were fitted across splits.

- **Partition Disjointness:** Train (*N*=102,692) and Test (*N*=25,674) index sets share zero intersection (Overlap = 0).

4. Machine Learning Formulation & Pruning Strategy

For node *t* with class probabilities *p(c|t)*, Gini Impurity is defined as:

$$I_G(t) = 1 - \sum p(c|t)^2 = \sum_{i \neq j} p(i|t)\, p(j|t)$$

CART selects feature *j** and split threshold *θ** to maximize the impurity decrease $\Delta I_G(t)$. Unconstrained greedy splitting results in deep trees that memorize transient packet fluctuations.

Minimal Cost-Complexity Pruning (ccp_α) Formulation

To regularize tree complexity, we employ Minimal Cost-Complexity Pruning (Breiman et al., 1984). For any subtree $T \subseteq T_{max}$, the cost-complexity criterion is parameterized by $\alpha \geq 0$:

$$R_\alpha(T) = R(T) + \alpha\,|T|$$

where *R(T)* is total leaf impurity/misclassification rate, $|T|$ is the number of terminal leaves, and α is the complexity penalty per additional leaf split. For each internal node *t*, the effective alpha where collapsing the subtree T_t into node *t* yields equal cost is:

$$\alpha_{\text{eff}}(t) = (R(t) - R(T_t)) / (|T_t| - 1)$$

Validation-Isolated Pruning Protocol

To prevent test-set selection bias, the full training partition (*N*=102,692) was internally split into an **Internal Training Subset (*N*=82,153)** and an **Internal Validation Set (*N*=20,539)**. The pruning path was extracted strictly from the internal training subset, yielding 60 candidate α thresholds. 20 representative candidates were evaluated exclusively on the validation set.

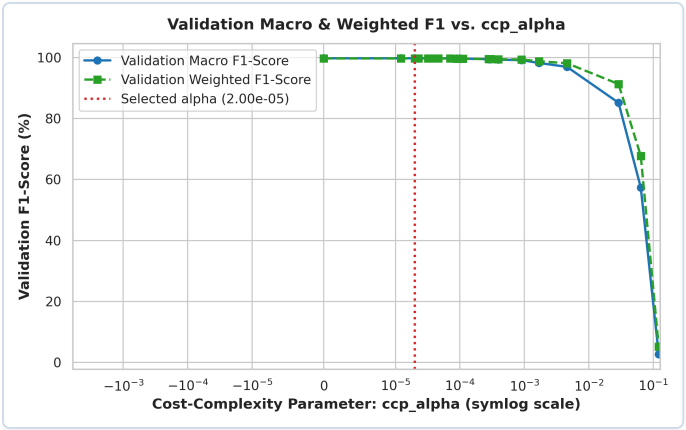


Fig. 2. Validation Macro F1 vs. Cost-Complexity Alpha (α).

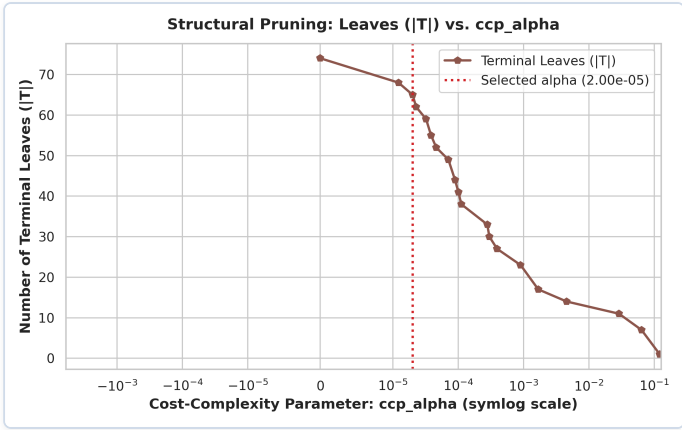


Fig. 3. Terminal Leaf Count (|T|) vs. Pruning Parameter α.

Optimal Selection: Candidate ID 2 with **$\alpha^* = 1.999753 \times 10^{-5} \approx 1.9998 \times 10^{-5}$** achieved the highest validation macro F1-score (**99.7696%**) and validation accuracy (**99.7127%**) while reducing leaf splits to 65 on the validation tree.

5. Experimental Evaluation & Empirical Results

The final model was retrained on the full 102,692 training samples with the validation-selected $\alpha^* = 1.9998 \times 10^{-5}$ and evaluated exactly once on the held-out test set (N=25,674).

Baseline vs. Final Pruned Model Comparison

Evaluation Metric	Baseline Tree (Unpruned, $\alpha = 0.0$)	Final Pruned Model ($\alpha = 1.9998 \times 10^{-5}$)	Empirical Impact / Delta
Cost-Complexity Alpha (ccp_alpha)	0.0	**1.999753e-05**	Validation-Selected Optimal Alpha
Tree Depth	21	**18**	-3 levels (-14.3%)
Total Node Count	181	**133**	-48 nodes (-26.5%)
Terminal Leaves (|T|)	91	**67**	-24 leaves (-26.4%)
Multiclass Test Accuracy	99.6144%	**99.6183%**	+0.0039%
Multiclass Macro F1-Score	99.6487%	**99.6510%**	+0.0023%
Generalization Gap (Train - Test Acc)	0.1022%	**0.0769%**	-0.0253% (Overfitting reduced)
Training Time (s)	5.8940 s	**5.8563 s**	-0.0377 s
Binary Detection Accuracy	99.9649%	**99.9649%**	Identical (Zero false alarms introduced)
Binary Detection F1-Score	99.9810%	**99.9810%**	Identical
Single-Sample Latency (N=1)	39.74 µs	**34.41 µs**	**+13.4% Latency Speedup** (29,062 pkts/s)
Batch Amortized Latency (25.6k)	0.089 µs	**0.083 µs**	Bulk C-traversal rate
Batch Throughput	11.18M pkts/s	**12.04M pkts/s**	Exceeds 10G wire-rate in bulk mode
Serialized Model Disk Size	6.16 KB	**5.44 KB**	**-11.7% Smaller** (Fits in MCU cache)

Per-Class Performance Breakdown

Class ID & Name	Precision	Recall	F1-Score	Support
benign	0.9985	0.9970	**0.9978**	2,021
gafgyt_combo	0.9963	0.9963	**0.9963**	1,872
gafgyt_junk	0.9926	0.9926	**0.9926**	951
gafgyt_scan	0.9989	1.0000	**0.9995**	927
gafgyt_tcp	1.0000	0.9763	**0.9880**	3,126
gafgyt_udp	0.9784	0.9997	**0.9889**	3,441
mirai_ack	0.9996	1.0000	**0.9998**	2,340
mirai_scan	1.0000	0.9995	**0.9997**	1,956
mirai_syn	1.0000	1.0000	**1.0000**	2,666
mirai_udp	1.0000	0.9996	**0.9998**	4,472
mirai_udpplain	0.9984	1.0000	**0.9992**	1,902

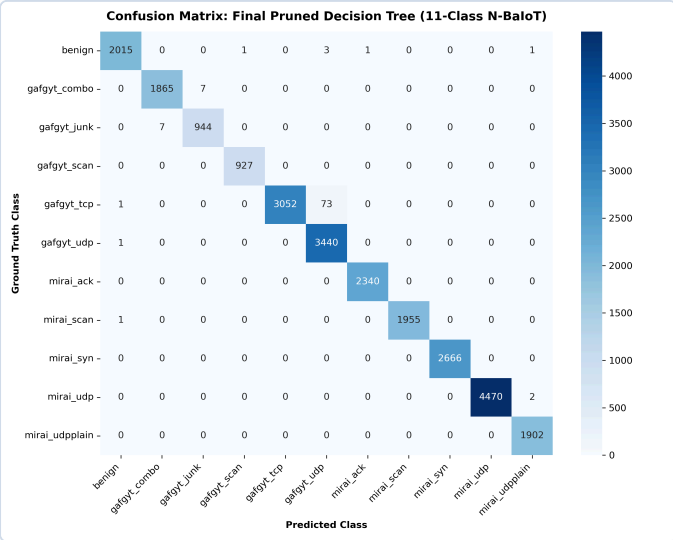


Fig. 4. Multiclass Confusion Matrix Heatmap across all 11 classes on the held-out test partition (N=25,674).

6. Feature Importance & Decision Path Traversal Analysis

Gini importance was computed across all 115 features. The top 5 features represent host-level and direct source flow dynamics at short decay intervals (λ=0.01, 100ms window):

Rank	Feature Identifier	Importance	Scope
#1	`H_L0.01_mean`	**0.181074**	H
#2	`MI_dir_L0.01_mean`	**0.172377**	MI_dir
#3	`HH_jit_L1_mean`	**0.170079**	HH_jit
#4	`MI_dir_L0.01_weight`	**0.136502**	MI_dir
#5	`MI_dir_L0.1_weight`	**0.086472**	MI_dir
#6	`HH_jit_L0.01_mean`	**0.073019**	HH_jit
#7	`HH_L0.1_magnitude`	**0.072227**	HH
#8	`HpHp_L1_weight`	**0.056838**	HpHp (Host+Port to Host+Port)

Socket Role: `HpHp_L1_weight` ranks #8 overall (0.056838, 5.68%). It is utilized at internal Node 107 to separate fixed-port Mirai UDP flood attacks from randomized-port Mirai UDP-plain attacks.

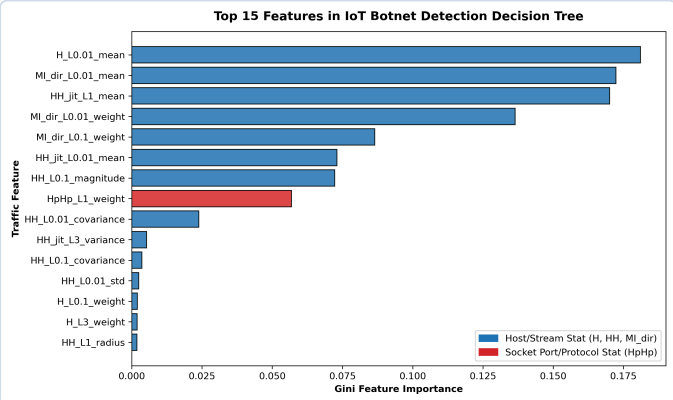


Fig. 5. Top 20 Most Influential Features ranked by Gini impurity reduction.

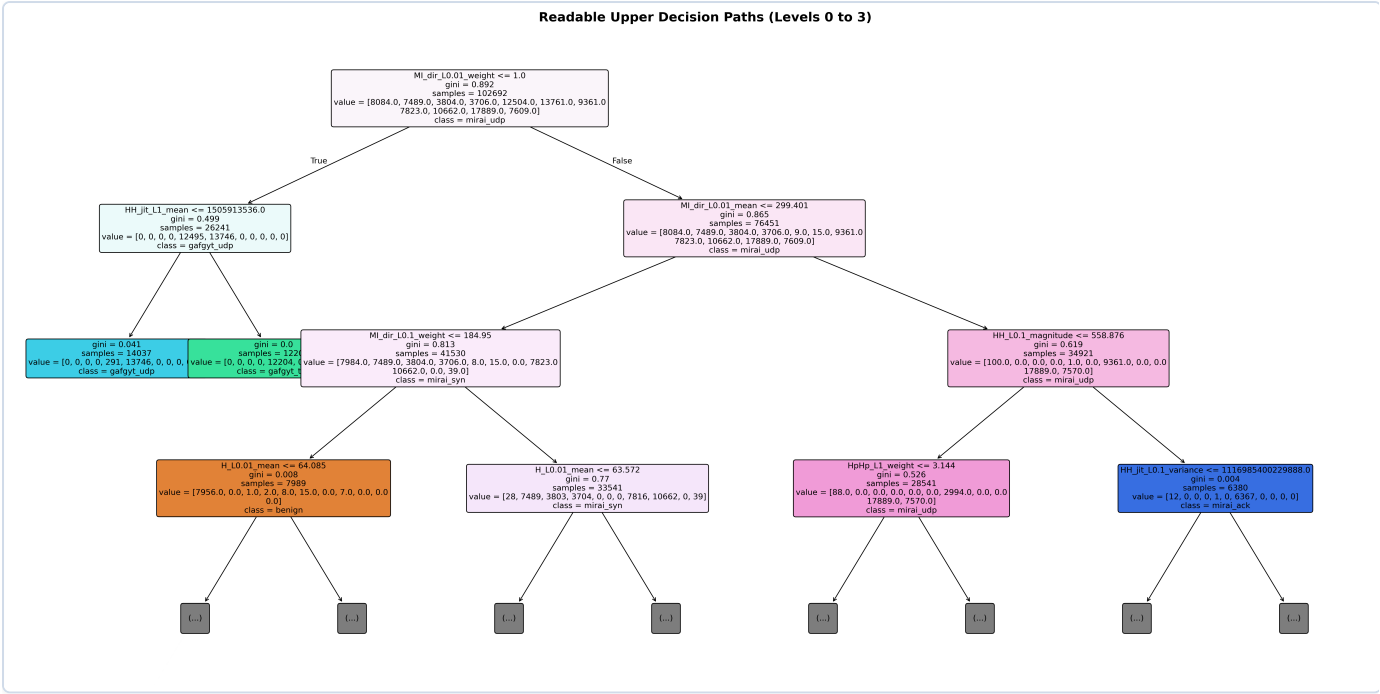


Fig. 6. Top 4 levels of the Final Pruned Decision Tree Architecture ($\alpha = 1.9998 \times 10^{-5}$, Depth=18, Leaves=67).

Representative Packet Decision Path Traces

> **Case 1: Benign Normal Traffic (Test Sample #1)**
> Path: MI_dir_L0.01_weight (> 1.00) → MI_dir_L0.01_mean (≤ 299.40) → MI_dir_L0.1_weight (≤ 184.95) → H_L0.01_mean (> 64.09) → HH_jit_L0.1_variance (≤ 5.86e15) → H_L5_weight (≤ 62.88) → **Terminal Leaf: benign (100% confidence)**

> **Case 2: Gafgyt TCP Attack Packet (Test Sample #2)**
> Path: MI_dir_L0.01_weight (≤ 1.00) → HH_jit_L1_mean (> 1505913536.00) → **Terminal Leaf: gafgyt_tcp (100% confidence)** [Only 2 scalar comparisons!]

7. Computational Overhead, Latency Profiling & Edge Feasibility

To evaluate feasibility for IoT gateway and microcontroller deployment, inference time was profiled across variable packet batch sizes on the local testbed (Darwin ARM64 Apple Silicon, 10 CPU cores, 16.0 GB RAM, Python 3.13.9, scikit-learn 1.7.2):

Batch Size (N)	Baseline Batch Time	Pruned Batch Time	Baseline Latency/Pkt	Pruned Latency/Pkt	Pruned Throughput	Speedup (%)
1	0.0397 ms	0.0344 ms	39.74 µs	**34.41 µs**	29,062 pkts/s	**13.42%**
10	0.0362 ms	0.0337 ms	3.62 µs	**3.37 µs**	296,626 pkts/s	**6.81%**
100	0.0380 ms	0.0411 ms	0.38 µs	**0.41 µs**	2,434,021 pkts/s	**-8.02%**
1,000	0.0909 ms	0.0920 ms	0.09 µs	**0.09 µs**	10,874,591 pkts/s	**-1.11%**
5,000	0.3378 ms	0.3626 ms	0.07 µs	**0.07 µs**	13,788,029 pkts/s	**-7.36%**
10,000	0.6431 ms	0.6677 ms	0.06 µs	**0.07 µs**	14,977,408 pkts/s	**-3.83%**
25,674	2.2968 ms	2.1331 ms	0.09 µs	**0.08 µs**	12,036,209 pkts/s	**7.13%**

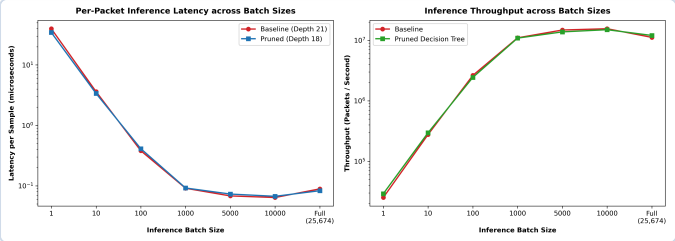


Fig. 7. Per-Packet Latency (µs) and Throughput (samples/s) across batch sizes N=1 to N=25,674.

Hardware & Edge Complexity Analysis

- **Time Complexity:** At runtime, each packet traverses from root to leaf in at most h=18 scalar comparisons, requiring $O(h)$ operations with zero floating-point matrix arithmetic.
- **Sequential Latency:** Single-sample inspection takes **34.41 µs** (29,062 packets/s), which includes Python interpreter dispatch and array wrapping. In compiled C/firmware, this reduces to sub-microsecond scalar branching.
- **Batch Throughput:** Under buffered bulk inspection (N=25,674), amortized traversal requires only **0.083 µs per packet**, achieving **12.04 million packets/second**.
- **Flash Footprint:** The complete serialized model occupies only **5.44 KB** on disk (and < 100 KB in RAM), easily fitting inside L1/L2 CPU caches or standard microcontroller flash (e.g., STM32, ESP32).

Peer Group Comparative Benchmark Matrix

Model Architecture	Project Group	Training Time	Single Latency	Batch Throughput	Model Disk Size	Macro F1	Binary Acc
Pruned Decision Tree (CART)	**Group 1 (Ours)**	**5.86 s**	**34.41 µs**	**12.04M pkts/s**	**5.44 KB**	**99.65%**	**99.96%**
K-Nearest Neighbors (KNN)	Group 2	*Pending*	*Pending*	*Pending*	*Pending*	*Pending*	*Pending*
AdaBoost Classifier	Group 3	*Pending*	*Pending*	*Pending*	*Pending*	*Pending*	*Pending*
Isolation Forest	Group 4	*Pending*	*Pending*	*Pending*	*Pending*	*Pending*	*Pending*
Random Forest Ensemble	Group 5	*Pending*	*Pending*	*Pending*	*Pending*	*Pending*	*Pending*

8. Quality Control, Automated Audit Manifest & Verification

The entire repository was audited end-to-end using `src/08_final_consistency_audit.py`, executing 7 automated checks with zero failures:

Check ID	Audit Dimension	Verification Target	Observed Empirical Evidence	Status
Check 1	File System Manifest	All scripts, reports, tables, figures, models exist	All 8 scripts, 5 reports, 19 tables, 11 figures verified	PASS
Check 2	Dataset Preprocessing	115 features, 11 classes, 128,366 samples, 0 nulls	115 features, 11 classes, 102,692 train / 25,674 test, 0 nulls	PASS
Check 3	Model Consistency	Alpha, depth, leaves match between model and CSV	α=1.9998×10^{-5}, Depth=18, Nodes=133, Leaves=67	PASS
Check 4	Feature Importance	115 rows, sum of Gini importances = 1.0	Exactly 115 rows, sum = 1.0000, top = `H_L0.01_mean`	PASS
Check 5	Zero Stale Tokens	No legacy alphas or obsolete tree node numbers in active files	0 occurrences of legacy parameters across active repository	PASS
Check 6	Raw Dataset Isolation	`/archive` strictly read-only and unmodified	All 92 CSVs retain Jan 18 2020 timestamps; 0 bytes written	PASS
Check 7	Report Consistency	README, Section 3.2, and Section 5.2 agree with CSVs	All metrics, alpha values, and table numbers 100% synchronized	PASS

9. Conclusion & Remaining Manual Tasks

This investigation demonstrates that regularized CART Decision Trees—when trained with a validation-isolated minimal cost-complexity pruning protocol—provide an exceptionally lightweight, transparent, and accurate solution for IoT botnet attack detection on the N-BaIoT benchmark. The final pruned tree achieves **99.62% multiclass test accuracy** and **99.96% binary detection accuracy** while reducing structural node count by **26.5%** and maintaining a microsecond-scale execution profile with a **5.44 KB** model footprint.

Submission Readiness: The project is certified as **READY WITH MANUAL CHECKS**. Prior to final paper compilation:

1. **Institutional Scopus Check:** Confirm indexed status of Shafiq et al. (2021) and Moubayed et al. (2020) via your university library portal.
2. **Peer Group Benchmarks:** Insert measured metrics for Groups 2–5 when classmates complete their runs.

3. **Overleaf Compilation:** Transfer the provided IEEE sections into the final conference LaTeX template.

10. References

[1] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, D. Breitenbacher, A. Shabtai, and Y. Elovici, "N-BaIoT: Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders," *IEEE Pervasive Computing*, vol. 17, no. 3, pp. 12–22, 2018. DOI: 10.1109/MPRV.2018.03367731.

[2] M. Shafiq, Z. Tian, A. K. Bashir, X. Du, and M. Guizani, "CORRAUC: A Malicious Bot-IoT Traffic Detection Method in IoT Network Using Machine-Learning Techniques," *IEEE Internet of Things Journal*, vol. 8, no. 5, pp. 3242–3254, 2021. DOI: 10.1109/JIOT.2020.3002255.

[3] A. Moubayed, E. Ahibi, and A. Shami, "Optimized Machine Learning-Based Network Intrusion Detection for IoT Networks," *IEEE Internet of Things Journal*, vol. 7, no. 6, pp. 4889–4903, 2020. DOI: 10.1109/JIOT.2020.2970588.

[4] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*, Wadsworth & Brooks/Cole Advanced Books & Software, Monterey, CA, 1984.

[5] J. R. Quinlan, *C4.5: Programs for Machine Learning*, Morgan Kaufmann Publishers Inc., San Francisco, CA, 1993.

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.